DAY 12 — ANSWERS

React Routing — Multi-Page React Application

Objective

Build a multi-page React application using React Router with Home, About, Students, and Contact pages. Navigation should work without a full page reload. The app also demonstrates a dynamic student route and a basic 404 page.

1. Key Concepts

Concept	Explanation
React Router	Library used to handle navigation between views in a React app.
BrowserRouter	Keeps the UI synchronized with the browser URL using the History API.
Routes and Route	Routes groups route definitions; Route maps a URL path to a component.
Link	Navigates to a route without a full browser page reload.
NavLink	Like Link, with support for styling the active route.
Dynamic route parameter	A variable path segment such as /students/:id.
404 page	Fallback page for unknown paths, usually configured with path="*".

2. Project Setup

Inside an existing Vite React project, run:

```
npm install react-router-dom
npm run dev
```

To create a new project, run: npm create vite@latest day12-react-routing -- --template react, then cd day12-react-routing, npm install, and npm install react-router-dom.

3. Suggested Folder Structure

```
src/
  components/Navbar.jsx
  pages/Home.jsx
  pages/About.jsx
  pages/Students.jsx
  pages/StudentDetails.jsx
  pages/Contact.jsx
  pages/NotFound.jsx
  App.jsx
  App.css
  main.jsx
```

4. Implementation

src/main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";
import "./App.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

src/App.jsx

```
import { Routes, Route } from "react-router-dom";
import Navbar from "./components/Navbar";
import Home from "./pages/Home";
import About from "./pages/About";
import Students from "./pages/Students";
import StudentDetails from "./pages/StudentDetails";
```

```
import Contact from "./pages/Contact";
import NotFound from "./pages/NotFound";

export default function App() {
  return (
    <>
      <Navbar />
      <main className="container">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/students" element={<Students />} />
          <Route path="/students/:id" element={<StudentDetails />} />
          <Route path="/contact" element={<Contact />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>
    </>
  );
}
```

src/components/Navbar.jsx

```
import { NavLink } from "react-router-dom";

export default function Navbar() {
  return (
    <nav className="navbar">
      <span className="brand">Student Portal</span>
      <div className="nav-links">
        <NavLink to="/">Home</NavLink>
        <NavLink to="/about">About</NavLink>
        <NavLink to="/students">Students</NavLink>
        <NavLink to="/contact">Contact</NavLink>
      </div>
    </nav>
  );
}
```

src/pages/Home.jsx

```
import { Link } from "react-router-dom";

export default function Home() {
  return <section className="card">
    <h1>Welcome to Student Portal</h1>
    <p>This is the Home page of our React Routing application.</p>
    <Link to="/students">View Students</Link>
  </section>;
}
```

src/pages/About.jsx

```
export default function About() {
  return <section className="card">
    <h1>About Us</h1>
    <p>This application demonstrates client-side routing with React Router.</p>
  </section>;
}
```

src/pages/Students.jsx

```
import { Link } from "react-router-dom";

const students = [
  { id: 1, name: "Aarav" },
  { id: 2, name: "Anaya" },
  { id: 3, name: "Rohan" }
];

export default function Students() {
  return <section className="card">
    <h1>Students</h1>
    <p>Select a student to view the dynamic route example.</p>
    <ul>
      {students.map(student => (
        <li key={student.id}>
          <Link to={`/students/${student.id}`}>{student.name}</Link>
        </li>
      ))}
    </ul>
  </section>;
}
```

src/pages/StudentDetails.jsx

```
import { useParams, Link } from "react-router-dom";

export default function StudentDetails() {
  const { id } = useParams();
```

```
  return <section className="card">
    <h1>Student Details</h1>
    <p>Selected student ID: {id}</p>
    <Link to="/students">Back to Students</Link>
  </section>;
}
```

src/pages/Contact.jsx

```
export default function Contact() {
  return <section className="card">
    <h1>Contact</h1>
    <p>Email: studentportal@example.com</p>
    <p>Phone: +91 98765 43210</p>
  </section>;
}
```

src/pages/NotFound.jsx

```
import { Link } from "react-router-dom";

export default function NotFound() {
  return <section className="card">
    <h1>404 — Page Not Found</h1>
    <p>Sorry, the page you are looking for does not exist.</p>
    <Link to="/">Return to Home</Link>
  </section>;
}
```

src/App.css

```
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f7fb;
  color: #243047;
}

.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 20px;
  flex-wrap: wrap;
  padding: 18px 7%;
  background: #244a7c;
  color: white;
}

.brand { font-size: 20px; font-weight: bold; }

.nav-links { display: flex; gap: 18px; flex-wrap: wrap; }

.nav-links a {
  color: white;
  text-decoration: none;
  padding: 8px 10px;
  border-radius: 6px;
}

.nav-links a.active, .nav-links a:hover {
  background: white;
  color: #244a7c;
}

.container { width: min(900px, 92%); margin: 40px auto; }

.card {
  padding: 28px;
  background: white;
  border-radius: 12px;
  box-shadow: 0 5px 20px rgba(25, 45, 80, 0.08);
}

.card h1 { color: #244a7c; }
.card a { color: #244a7c; }

@media (max-width: 600px) {
  .navbar { align-items: flex-start; flex-direction: column; }
  .nav-links { gap: 8px; }
  .container { margin: 24px auto; }
  .card { padding: 20px; }
}
```

5. How It Works

• BrowserRouter enables URL-based routing. Routes and Route decide which page component to display.

• NavLink creates navigation links and supports active-link styling. Link navigates without a full page reload.

• The /students/:id path contains a dynamic parameter. useParams() reads the ID from the URL.

• The path="*" route catches unknown URLs and displays the 404 page.

6. Testing and Expected Output

1. Run npm run dev and open the local URL shown in the terminal.

2. Click Home, About, Students, and Contact in the navigation bar. Each page should change without a full page reload.

3. On Students, click a name to open a route such as /students/1 and view the dynamic ID.

4. Open an unknown path such as /random-page. The 404 page should appear.

5. Resize the browser to check the responsive layout.

Conclusion

This project demonstrates client-side navigation, multiple page components, dynamic route parameters, and a fallback 404 page. Student entries are sample frontend data and are not stored in a database.